API Reference

This document is the user-facing API reference for SoCLaaS as currently implemented.

SoCLaaS exposes an OpenAI-compatible gateway API for inference plus a separate self-service portal API for API-key lifecycle and budget visibility.

Authentication

Gateway requests to `/v1/*` require:

```
Authorization: Bearer <soclaas-api-key>
```
Copy

SoCLaaS API keys use this format:

```
clsk_<prefix>_<secret>
```
Copy

If the bearer token is missing, the gateway returns `401` with:

```
{
  "error": "missing bearer token"
}
```
Copy

If the API key is invalid or revoked, the gateway returns `401` with:

```
{
  "error": "invalid API key"
}
```
Copy

List of Endpoints

- GET /v1/models
- POST /v1/chat/completions
- POST /v1/responses
- POST /v1/embeddings
- POST /v1/audio/transcriptions

Only the `/v1/*` endpoints are intended for application users.

Common Gateway Behavior

- Public model names are operator-managed aliases. Clients should use the `id` values returned by `GET /v1/models`.
- Model access rules apply both to `GET /v1/models` and to generation requests. If a key cannot use a model, it will not appear in the model list.
- Requests may be rejected with `429` for either rate limits or budget limits.
- Daily and monthly spend windows reset on UTC boundaries.
- Usage and accounting are recorded against the authenticated SoCLaaS API key.

API Endpoints

GET /v1/models

Returns the model catalog visible to the calling API key.

Example:

```
curl \
  -H "Authorization: Bearer $SOCLAAS_API_KEY" \
  $SOCLAAS_BASE_URL/models
```
Copy

Example response:

```
{
  "object": "list",
  "data": [
    {
      "id": "llama3.1:8b",
      "object": "model",
      "created": 0,
      "owned_by": "soclaas",
      "context_length": 65536,
      "context_window": 65536,
      "max_context_tokens": 65536,
      "max_model_len": 65536,
      "soclaas": {
        "display_name": "Llama 3.1 8B",
        "description": "General-purpose internal chat model."
      }
    }
  ]
}
```
Copy

Notes:

- Standard OpenAI-style model fields are preserved.
- SoCLaaS-specific catalog metadata is namespaced under `soclaas`.
- `soclaas.display_name` and `soclaas.description` may be blank if operators have not set them

POST /v1/chat/completions

This is the primary low-transformation inference endpoint. It is intended to behave like the OpenAI Chat Completions API while routing to SoCLaaS-managed backends.

Minimum request:

```
{
  "model": "llama3.1:8b",
  "messages": [
    {"role": "user", "content": "Say hello in one sentence."}
  ]
}
```
Copy

Example:

```
curl $SOCLAAS_BASE_URL/chat/completions \
  -H "Authorization: Bearer $SOCLAAS_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "llama3.1:8b",
    "messages": [
      {"role": "system", "content": "Be concise."},
      {"role": "user", "content": "Say hello in one sentence."}
    ]
  }'
```
Copy

Streaming is supported with standard server-sent events:

```
{
  "model": "llama3.1:8b",
  "messages": [
    {"role": "user", "content": "Stream a short answer."}
  ],
```

```
  "stream": true
}
```
Copy

Notes:

- `model` is required. Missing `model` returns `400`.
- For streaming requests, SoCLaaS ensures upstream `stream_options.include_usage=true`.
- For non-streaming requests, `stream_options` is stripped before proxying upstream.
- This path does not execute tools in the gateway. It is the best choice for plain chat-completions clients.

POST /v1/responses

This endpoint is a compatibility shim for clients that expect the OpenAI Responses API. Internally, SoCLaaS translates the request into chat-completions traffic and translates the result back into Responses-shaped JSON or SSE.

Minimum request:

```
{
  "model": "llama3.1:8b",
  "input": "Say hello in one sentence."
}
```
Copy

Example:

```
curl $SOCLAAS_BASE_URL/responses \
  -H "Authorization: Bearer $SOCLAAS_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "llama3.1:8b",
    "input": "Summarize SoCLaaS in one sentence."
  }'
```
Copy

Example response:

```
{
  "id": "resp_123",
  "object": "response",
  "created_at": 1783900000,
  "status": "completed",
  "model": "llama3.1:8b",
  "output": [
    {
      "id": "msg_123",
      "type": "message",
      "status": "completed",
      "role": "assistant",
      "content": [
        {
          "type": "output_text",
          "text": "SoCLaaS is an OpenAI-compatible gateway for controlled access to internal LLM backends.",
          "annotations": []
        }
      ]
    }
  ],
  "output_text": "SoCLaaS is an OpenAI-compatible gateway for controlled access to internal LLM backends.",
  "usage": {
    "input_tokens": 12,
    "output_tokens": 18,
    "total_tokens": 30
  }
}
```
Copy

Supported request fields:

- model
- input
- instructions

- stream
- max_output_tokens
- temperature
- top_p
- tools
- tool_choice
- parallel_tool_calls
- previous_response_id

Current limitations:

- `background=true` is not supported and returns `400`.
- Response state is not durable. `previous_response_id` is kept in one gateway process only.
- The in-memory response-state cache currently has a 15-minute TTL and does not survive gateway restarts.
- Persisted response retrieval, response cancellation, and hosted OpenAI built-in tools are not implemented.

Responses Input Rules

- String `input` is treated as one user message.
- `nstructions ''becomes a system message. * Array ''input` is supported for message-style items and tool outputs.
- `revious_response_id` can be used for follow-up turns after tool execution, but only within the temporary gateway cache window.

If `previous_response_id` is unknown or expired, the gateway returns `400`.

Responses Tool Behavior

SoCLaaS supports two categories of tools on `/v1/responses`:

- Client-executed tools, which are returned to the caller for local execution.
- Gateway-executed web tools, when enabled and permitted.

Client-executed tools:

- OpenAI-style `function` tools
- `custom` tools
- `shell` and `local_shell`
- `apply_patch`

These are translated through the shim, but SoCLaaS does not execute them.

Gateway-executed web tools:

- web_fetch
- web_search
- web_search_preview

These tools are optional. They work only when:

- - the gateway has the feature enabled globally, and
- - the authenticated API key policy allows the tool

If a client forces a disabled or disallowed web tool through `tool_choice`, SoCLaaS returns `403`.

POST /v1/embeddings

Creates vector embeddings through a SoCLaaS-managed embedding backend. The endpoint accepts and returns the OpenAI Embeddings API shape; SoCLaaS passes the compatible request fields and the upstream response through without transforming the embedding vectors.

`model` must be an active public model ID returned by `GET /v1/models`, be permitted by the API key, and have the `embeddings` capability. The gateway resolves aliases and rewrites the public model ID to its configured provider model before dispatching the

request. A chat-only model cannot be used for embeddings.

Required fields:

- `model`: the public embedding model ID.
- `input`: one input string or an array of input strings. Token-ID input is also forwarded when supported by the selected backend.

Common optional OpenAI-compatible fields are forwarded unchanged:

- `encoding_format` (for example, `float` or `base64`)
- `dimensions`
- `user`

Support for optional fields, accepted input sizes, vector dimensions, and the embedding encoding ultimately depends on the selected backend and model.

Example with one input:

```
curl --fail-with-body \
  -X POST https://gateway.example.com/v1/embeddings \
  -H "Authorization: Bearer $SOCLAAS_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "core-embeddings",
    "input": "SoCLaaS provides controlled access to LLM backends.",
    "encoding_format": "float"
  }'
```
Copy

Example with a batch of inputs:

```
{
  "model": "core-embeddings",
  "input": [
    "First document to index.",
    "Second document to index."
  ]
}
```
Copy

Example response:

```
{
  "object": "list",
  "data": [
    {
      "object": "embedding",
      "index": 0,
      "embedding": [0.0123, -0.0456, 0.0789]
    }
  ],
  "model": "nomic-embed-text",
  "usage": {
    "prompt_tokens": 9,
    "total_tokens": 9
  }
}
```
Copy

The response `model` field is supplied by the upstream backend and can contain the provider model ID rather than the SoCLaaS public model ID. Do not infer model access from that field; use `GET /v1/models` for the public catalog.

SoCLaaS applies API-key rate and concurrency limits, dispatch queueing, backend capacity controls, timeout handling, and model pricing/accounting to embedding requests. When an upstream response provides token usage, the gateway records it for accounting.

POST /v1/audio/transcriptions

Transcribes one audio file through a SoCLaaS-managed `whisper` backend. The endpoint is OpenAI-compatible and accepts `multipart/form-data` rather than a JSON body.

The requested `model` is a SoCLaaS public model. It must be active, permitted by the API key, and have the `audio-transcription` capability. The gateway rewrites that field to the configured provider model before forwarding the request. For example, an operator can map public model `whisper-large-v3` to provider model `Systran/faster-whisper-large-v3`.

Required fields:

- `file`: one audio file upload.
- `model`: the public model ID returned by `GET /v1/models`.

Common optional fields are forwarded unchanged to the Whisper backend:

- `language`
- `prompt`
- `response_format` (`json`, `text`, `verbose_json`, `srt`, or `vtt`, when supported by the backend)
- `temperature`
- `timestamp_granularities[]`
- `stream`

Example:

```
SOCLAAS_BASE_URL="https://gateway.example.com"
SOCLAAS_API_KEY="clsk_your_api_key"
SOCLAAS_MODEL="whisper-large-v3"
SOCLAAS_AUDIO_FILE="./recording.mp3"

curl --fail-with-body \
  -X POST "${SOCLAAS_BASE_URL}/v1/audio/transcriptions" \
  -H "Authorization: Bearer ${SOCLAAS_API_KEY}" \
  -F "model=${SOCLAAS_MODEL}" \
  -F "file=@${SOCLAAS_AUDIO_FILE}" \
  -F "language=en" \
  -F "response_format=json"
```
Copy

Example JSON response:

```
{
  "text": "Transcribed speech appears here."
}
```
Copy

The gateway applies its existing request-body limit (50 MiB by default), API-key rate and concurrency limits, queueing, backend capacity controls, and backend timeout. It records byte counts and request metadata but does not token-charge transcription responses that do not contain token usage.

Streaming Transcriptions

When the selected faster-whisper-server supports it, set `stream=true` to receive its server-sent events without response transformation:

```
curl --no-buffer --fail-with-body \
  -X POST "${SOCLAAS_BASE_URL}/v1/audio/transcriptions" \
  -H "Authorization: Bearer ${SOCLAAS_API_KEY}" \
  -H "Accept: text/event-stream" \
  -F "model=${SOCLAAS_MODEL}" \
  -F "file=@${SOCLAAS_AUDIO_FILE}" \
  -F "stream=true"
```
Copy

Common client errors are `400` for a missing `model`, missing or multiple `file` parts, malformed multipart data, or a model without the `audio-transcription` capability; `403` for a model denied by policy; and `413` when the upload exceeds the gateway limit.

Common Gateway Errors

Typical gateway error responses use this shape:

```
{
  "error": "message"
}
```
Copy

Common statuses:

- 400 Bad Request: Invalid JSON, missing `model`, unsupported Responses feature, expired `previous_response_id.`
- 401 Unauthorized: Missing bearer token, invalid API key.
- 403 Forbidden: Model not allowed, username mapping failed, forced web tool not allowed.
- 429 Too Many Requests: Rate limit exceeded, quota exceeded.
- 503 Service Unavailable: This is a server-side issue, with common reasons such as authentication backend unavailable, policy enforcement unavailable, and web search provider not configured.

Related

- Get Access
- API Reference
- Quick Start Guide
- Using with a GUI

Ask SoChat

SoChat

Hello! I'm SoChat, here to assist you with information, guides, and services from SoC's Computing Facilities. What are you looking for?

Message... (paste an image to attach it)

SoChat is powered by SoCLaaS. It may not always provide accurate or complete information, so please verify important details independently. Your feedback is welcome and will help us improve SoChat.